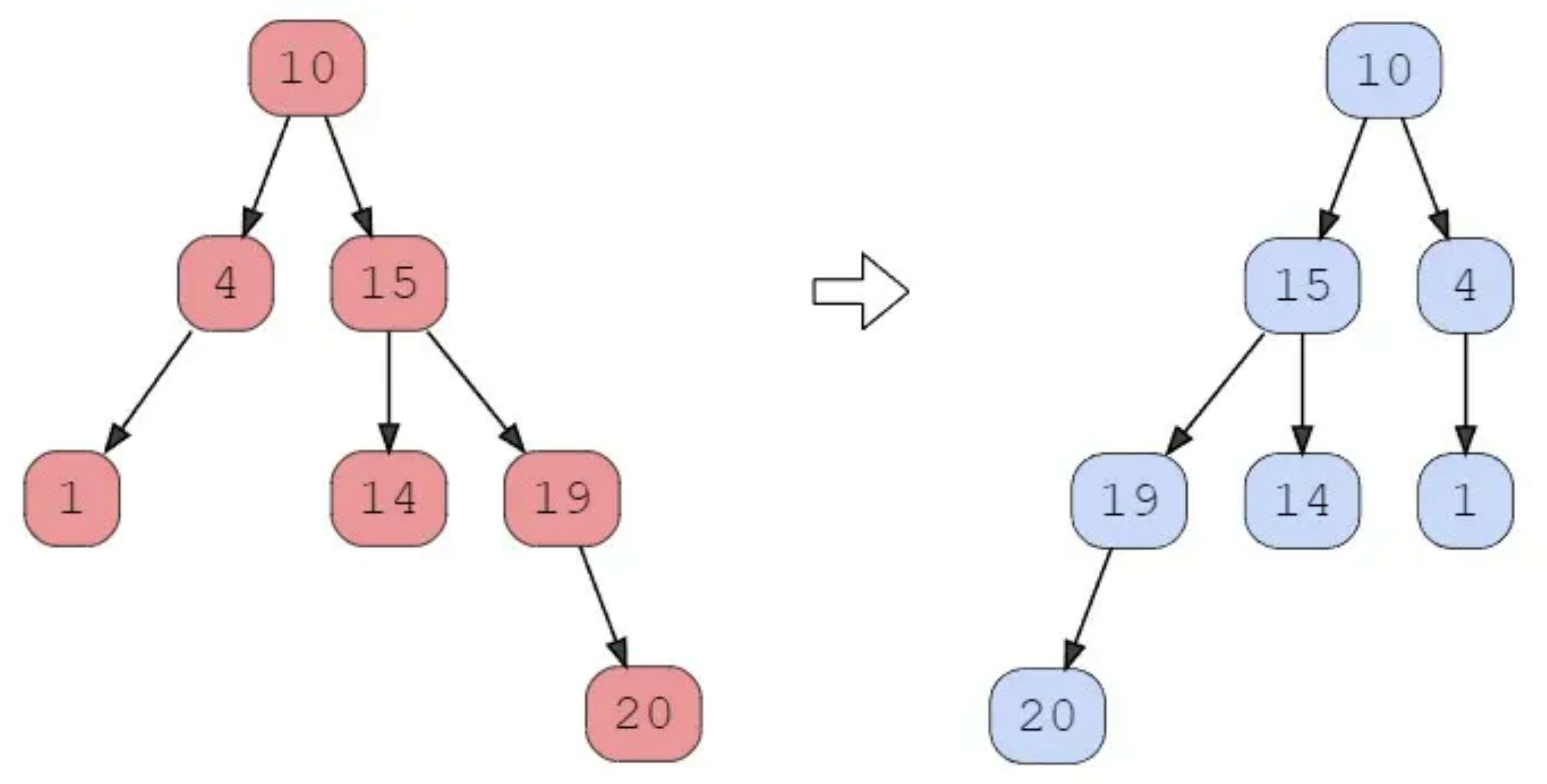
Solution: Invert Binary Tree

Problem Statement

Given the root of a binary tree, invert it.

Example:

Given the following two binary trees:

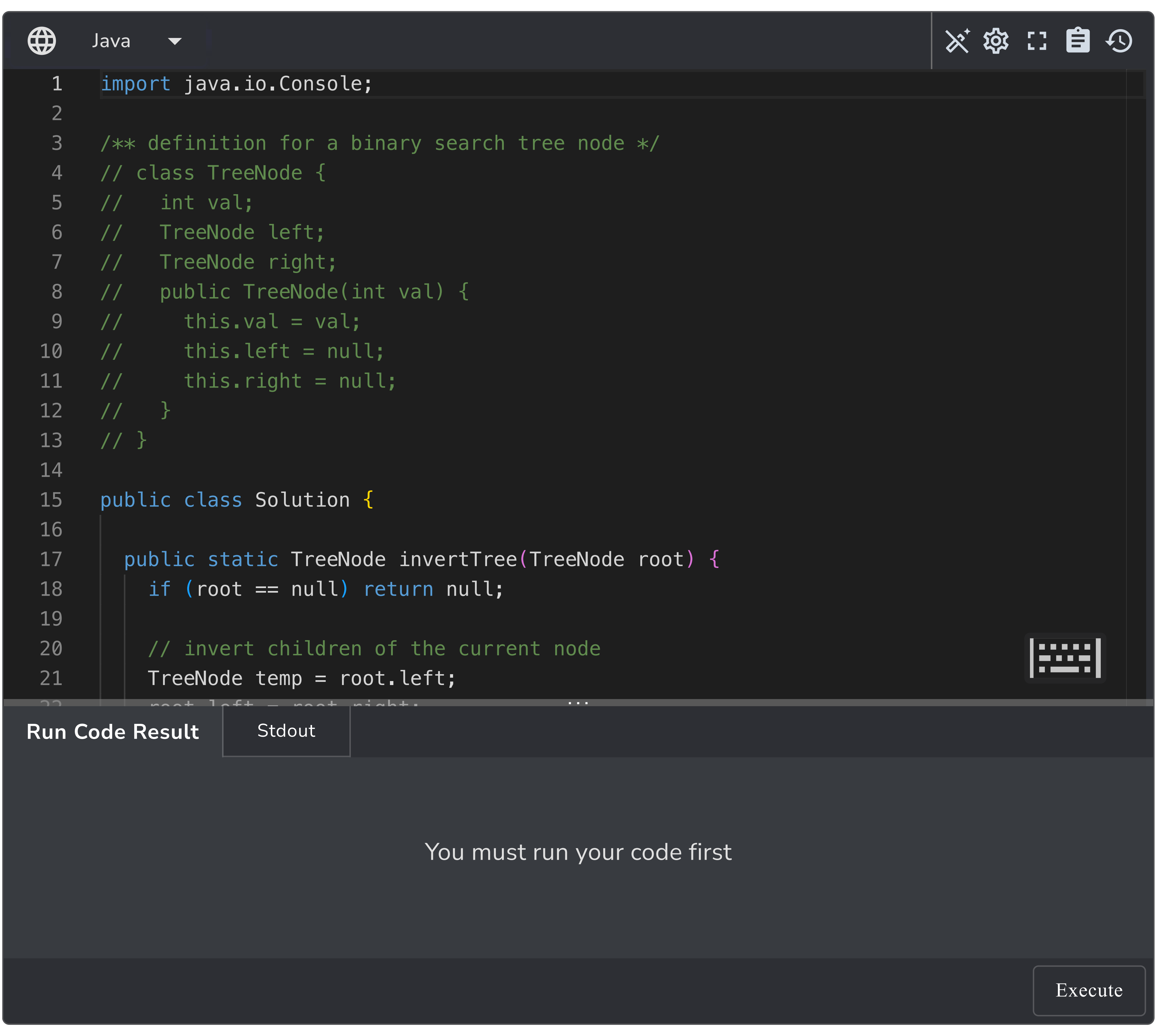


Solution

This problem is quite similar to `Same Tree`. We can follow the same approach. After swapping left and right child of a node, we will recursively invert its left and right subtrees.

Code

Here is what our algorithm will look like:



```java
import java.io.Console;

/** definition for a binary search tree node */
// class TreeNode {
//   int val;
//   TreeNode left;
//   TreeNode right;
//   public TreeNode(int val) {
//     this.val = val;
//     this.left = null;
//     this.right = null;
//   }
// }

public class Solution {

  public static TreeNode invertTree(TreeNode root) {
    if (root == null) return null;

    // invert children of the current node
    TreeNode temp = root.left;
```

You must run your code first

Time Complexity

Since we traverse each node once, the solution will take O(N) time where 'N' is the total number of nodes in the tree.

Space Complexity

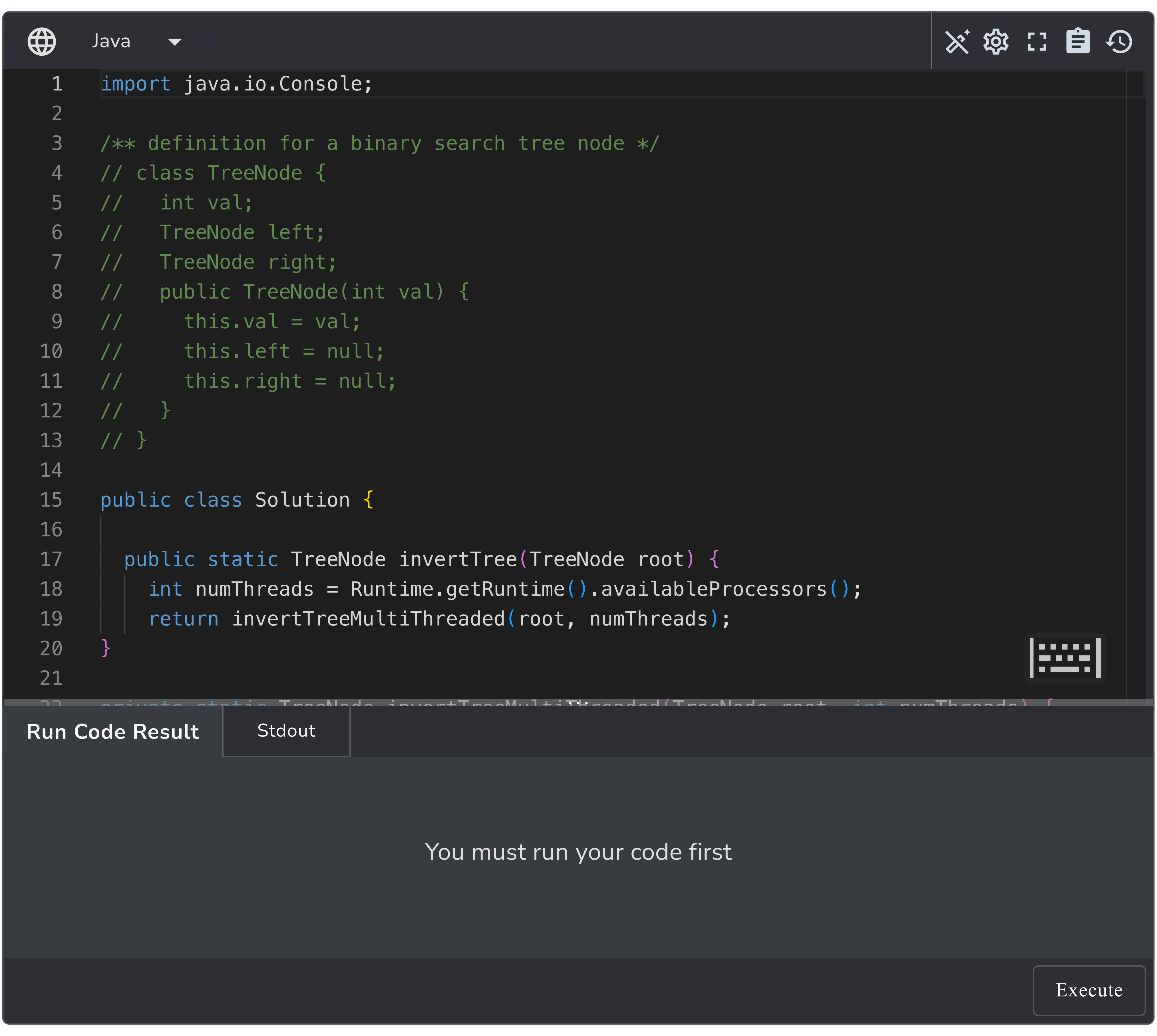
We will need O(N) space for the recursion stack in the worst case (when the binary tree is a list). Overall, we will need to store nodes equal to the height of the tree, hence, O(H) space complexity, where H is the height of the given tree.

Making the Algorithm Multi-threaded

To further improve the algorithm, we can make **invertTree**() multi-threaded to invert left and right subtrees in separate threads.

We can find how many cores the machine has on which our algorithm is running. We will, then, start multiple threads so that each core can run one thread.

Here is the code that takes care of this scenario:



```java
import java.io.Console;

/** definition for a binary search tree node */
// class TreeNode {
//   int val;
//   TreeNode left;
//   TreeNode right;
//   public TreeNode(int val) {
//     this.val = val;
//     this.left = null;
//     this.right = null;
//   }
// }

public class Solution {

  public static TreeNode invertTree(TreeNode root) {
    int numThreads = Runtime.getRuntime().availableProcessors();
    return invertTreeMultiThreaded(root, numThreads);
}

```

You must run your code first

Time and Space Complexities

Everything has the same complexity as the previous solution.

← Previous
Invert Binary Tree (Medium)

Next →
Binary Search Tree Iterator (Medium)

☐ Mark as Completed